

This article describes in additional detail the format of the **Chunk Data packet**.

Contents [hide]
1 Concepts 1.1 Chunks columns and Chunk sections 1.2 Global palettes 1.2.1 Global block state palette 1.2.2 Biome registry 1.3 Local palettes 1.4 Notes 2 Packet structure 2.1 Heightmap structure 2.2 Data structure 2.2.1 Chunk Section structure 3 Paletted Container structure 3.1 Palette formats 3.1.1 Single valued 3.1.2 Indirect 3.1.3 Direct 3.1.4 Example 3.2 Data Array format 3.2.1 Visual example 3.2.2 Hints for implementers 4 Tips and notes 5 Full implementations 6 Sample data 6.1 Old format

Concepts [edit | edit source]

Chunks columns and Chunk sections [edit | edit source]

You've probably heard the term "chunk" before. Minecraft uses chunks to store and transfer world data. However, there are actually 2 different concepts that are both called "chunks" in different contexts: chunk columns and chunk sections.

A **chunk column** is a collection of blocks with a horizontal size of 16×16, spanning the entire buildable area on the vertical axis. This is what most players think of when they hear the term "chunk". However, these are not the smallest unit data is stored in the game; chunk columns are vertically divided into chunk sections, each 16 blocks tall.

Chunk columns store block entities, entities, tick data, and an array of sections.

A **chunk section** is a 16×16×16 collection of blocks (chunk sections are cubic). This is the actual area that blocks are stored in, and is often the concept Mojang refers to via "chunk". Breaking columns into sections wouldn't be useful, except that you don't need to send all chunk sections in a column: if a section is empty, then it doesn't need to be sent (more on this later).

Chunk sections store blocks, biomes and light data (both block light and sky light). Additionally, they can be associated with at most two **local palettes**—one for blocks, one for biomes. A chunk section can contain at maximum 4096 (16×16×16, or 2¹²) unique block state IDs, and 64 (4×4×4) unique biome IDs (but, it is highly unlikely that such a section will occur in normal circumstances).

Chunk columns and chunk sections are both displayed when chunk border rendering is enabled (F3 ⇧ G). Chunk columns borders are indicated via the red vertical lines, while chunk sections borders are indicated by the blue lines.

Global palettes [edit | edit source]

The global palettes map block states and biomes to protocol-wide numeric identifiers.

Global block state palette [edit | edit source]

The global block state palette is generated based on the minecraft:block registry, which in turn is hardcoded into Minecraft, and can only be changed via modding. Such changes break protocol compatibility, and as such, modding frameworks typically include protocol extensions to negotiate which IDs the client and server have in common.

One block state ID is allocated for each unique block state of a block; if a block has multiple properties then the number of allocated states is the product of the number of values for each property. The block state IDs belonging to a given block are always consecutive. Other than that, the ordering of block states is hardcoded, and somewhat arbitrary.

The Data Generators system can be used to generate a list of all block state IDs.

Biome registry [edit | edit source]

The global palette for biomes is the minecraft:worldgen/biome synchronized registry, which is defined at runtime in a Registry Data packet sent by the server during the Configuration phase.

The vanilla server pulls these biome definitions from data packs.

Local palettes [edit | edit source]

A local palette maps a smaller set of IDs within a **chunk section** to global palette IDs. Other than skipping empty sections, correct use of palettes is the biggest place where data can be saved. For example, encoding any of the IDs in the global block state palette as of vanilla 1.20.2 requires 15 bits. Given that most sections contain only a few different blocks, using 15 bits per block to represent a chunk section that is only stone, gravel, and air would be extremely wasteful. Instead, a list of global palette IDs is sent to define the local palette (for instance, 40 57 0), and indices into that list are sent as the block state or biome values within the chunk (so 40 57 0 would be sent as 0 57 1, and 0 as 2).^[*concept note*]

The number of bits used to encode local palette indices varies based on the number of indices, and the global palette in question. If a threshold on the number of unique IDs in the section is exceeded, a local palette is not used, and global palette IDs are used directly instead.

The concept of palettes is more commonly used with colors in an image; Wikipedia's articles on color look-up tables, indexed colors, and palettes in general may be helpful for fully grokking it.

 ⚠ Note that the notchian client (and server) store their chunk data within the compacted, paletted format. Sending non-compacted data not only wastes bandwidth, but also leads to increased memory use clientside; while this is OK for an initial implementation it is strongly encouraged that one compacts the block data as soon as possible.

Notes [edit | edit source]

- ↑ There is no requirement for the global palette IDs listed in a local palette to be *monotonic*; the order within the list is entirely arbitrary and often has to do with how the palette is built (if it finds a stone block before an air block, stone can come first). (However, although the order of the palette entries can be arbitrary, it can theoretically be optimized to ensure the maximum possible DEFLATE compression. This optimization offers little to no gain, so generally do not attempt it.) However, there shouldn't be any gaps in the palette, as gaps would increase the size of the palette when it is sent.

Packet structure [edit | edit source]

Packet ID	State	Bound To	Field Name	Field Type	Notes
0x27	Play	Client	Chunk X	 Int	Chunk coordinate (block coordinate divided by 16, rounded down).
			Chunk Z	 Int	Chunk coordinate (block coordinate divided by 16, rounded down).
			Heightmaps	 Prefixed Array of Heightmap	See #Heightmap structure below.
			Size	 VarInt	Size of Data in bytes; in some cases this is larger than it needs to be (e.g. MC-131684 ^{[?]} , MC-247438 ^{[?]}) in which case extra bytes should be skipped before reading fields after Data.
			Data	 Byte Array	See #Data structure below.
			Additional Data	Various	See Protocol#Chunk Data and Update Light .

Heightmap structure [edit | edit source]

Minecraft uses heightmaps to optimize various operations on both the server and the client. All heightmaps encode the position of the highest "occupied" block in each column of blocks within a chunk column. The differences have to do with which blocks are considered to be "occupied".

Rather than calculating them from the chunk data, the client receives the initial heightmaps it needs from the server. This trades an increase in network usage for a decrease in client-side processing. Once a chunk is loaded, the client updates its heightmaps based on block changes independently from the server.

No heightmaps are strictly required for the client to accept a chunk. If a heightmap is missing from a Chunk Data packet, the client will initialize it with all heights set to their minimum values. However, block changes will still cause the corresponding height values to be updated as normal.

As of 1.21.5, the Heightmap structure is as follows.

Heightmap Structure		
Field Name	Field Type	Notes
Type	 VarInt Enum	See the table below.
Data	 Prefixed Array of Long	Packed data array. Described below.

The height values of a heightmap are packed into the long array in the same manner described in #Data Array format, and ordered such that the fastest-increasing coordinate is x. (However, there are only 256 entries—one for each block column.) The Bits Per Entry value used is calculated as ceil(log₂(*world_height* + 1)). This is because the number of possible height values is one more than the world height—ranging from 0 (completely blank column; not even bedrock) to world height (highest position is occupied). Note that this means, for example, that a world with height 256 will use a Bits Per Entry of 9. Since the minimum world height might be negative, it must be added as an offset to get the actual highest occupied Y coordinate.

The following heightmaps are currently used by the client:

Name	ID	Considers Occupied	Purposes
WORLD_SURFACE	1	All blocks other than air, cave air and void air.	To determine if a beacon beam is obstructed.
MOTION_BLOCKING	4	"Solid" blocks, except bamboo saplings and cactuses; fluids.	To determine where to display rain and snow.
MOTION_BLOCKING_NO_LEAVES	5	Same as MOTION_BLOCKING, excluding leaf blocks.	

This list is exhaustive as of 1.21.8. The listed purposes appear to be exhaustive as of 1.20.2.

Data structure [edit | edit source]

The data section of the packet contains most of the useful data for the chunk.

Field Name	Field Type	Notes
Data	 Chunk Section	This array is NOT length-prefixed. The number of elements in the array is calculated based on the world's height. Sections are sent bottom-to-top. Starting with 1.18, the world height changes based on the dimension. The height of each dimension is assigned by the server in its corresponding entry in the minecraft:dimension_type synchronized registry . For example, the vanilla overworld is 384 blocks tall, meaning 24 chunk sections will be included in this array.

Chunk Section structure [edit | edit source]

⚠ **This section needs to be updated.**

Please update this section to reflect recent updates or newly available information. The talk page may contain suggestions.

Reason: How do biomes work now? The biome change happened at the same time as the seed change, but it's not clear how/if biomes could be computed given that it's not the actual seed...^[*if/mojira discussion*] which notes that it seems to be some kind of interpolation)

A Chunk Section is defined in terms of other **data types**. A Chunk Section consists of the following fields:

Field Name	Field Type	Notes
Block count	 Short	Number of non-air blocks present in the chunk section. "Non-air" is defined as any fluid and block other than air, cave air, and void air. The client will keep count of the blocks as they are broken and placed, and, if the block count reaches 0, the whole chunk section is not rendered, even if it still has blocks.
Block states	 Paletted Container	Consists of 4096 entries, representing all the blocks in the chunk section.
Biomes	 Paletted Container	Consists of 64 entries, representing 4×4×4 biome regions in the chunk section.

Paletted Container structure [edit | edit source]

A Paletted Container is a palette-based storage of entries. Paletted Containers have an associated global palette (either block states or biomes as of now), where values are mapped from. A Paletted Container consists of the following fields:

Field Name	Field Type	Notes
Bits Per Entry	 Unsigned Byte	Determines how many bits are used to encode entries. Note that not all numbers are valid here.
Palette	Varies	See #Palette formats below.
Data Array	 Array of Long	See #Data Array format below.

Palette formats [edit | edit source]

The Bits Per Entry value determines what format is used for the Palette field, which in turn determines how values in the Data Array map to the global palette.

 ⚠ Values not listed in the following table are rounded upwards to the next one specified, or downwards if larger than the value for Direct. Therefore such values will lead to unexpected results, and should not be used.

There are currently three possible palette formats:

BPE (blocks)	BPE (biomes)	Palette Format
0	0	 Single valued
4-8	1-3	 Indirect
15+*	7+*	 Direct

*The Notchian client calculates the Bits Per Entry values for the Direct palette format at runtime based on the sizes of the global block state and biome palettes. As such, the value used for biomes is entirely dependent on the contents of the biome registry sent in the Registry Data packet; the value shown is only valid for vanilla servers with no custom data packs. If the BPE requirement for Direct is less than or equal to the maximum for Indirect, Direct will never be used given BPE values within the valid range.

**Similarly, if a sufficiently large number of blocks is added with mods, the value will be increased to compensate for the increased ID count. This increase can go up to 31 bits per entry (since registry IDs are signed integers). In case of Minecraft Forge, you can get the number of blocks with the "Number of ids" field found in the Registry Data packet in the Forge Handshake.

Single valued [edit | edit source]

When this palette format is used, the Data Array sent/received is empty, since entries can be inferred from the palette's single value.

Field Name	Field Type	Notes
Value	 VarInt	ID of the entry in the global palette.

Indirect [edit | edit source]

This is an actual palette which lists the entries used. Values in the Data Array are indices into the local palette, which in turn gives a proper global palette ID.

Field Name	Field Type	Notes
Palette Length	 VarInt	Number of elements in the following array.
Palette	 Array of VarInt	Mapping of IDs in the global palette to indices of this array.

Direct [edit | edit source]

Global palette IDs are stored directly as entries in the Data Array.

Field Name	Field Type	Notes
<i>no fields</i>		

Example [edit | edit source]

Here is an example showing a Chunk Section using a single-valued palette for block states, and an indirect palette with 2 indices for biomes:

 80 00 00 00 01 02 27 03 CC FF CC FF CC FF CC FF
The first bytes 80 00 are the number of non-air blocks in the chunk. They are followed by the Bits Per Entry 00, which is zero so we know the palette will have one element (not prefixed with length). This single element is the block state ID of air, 00.

The second part of the packet is for biomes. The first byte is their Bits Per Entry 01, followed by the length of the palette 02 and the two elements 27 03. The indexed data of this biome has 1 long element, which are 8 bytes each, giving the long CC FF CC FF CC FF CC FF.

Data Array format [edit | edit source]

As of 1.21.5, the length of the data array is no longer sent with the packet, but is instead calculated from the bits per entry and the number of entries.

The Data Array stores entries as Bits Per Entry–bit integers, corresponding to either local or global palette indices depending on the palette format in use. If Bits Per Entry is 0, it is empty.

Entries are stored in order of increasing x coordinate, within rows at increasing z coordinates, within layers at increasing y coordinates. In other words, the x coordinate increases the fastest, and the y coordinate the slowest.

A single long of the array holds several entries. The entries are tightly packed within the long, with the first entry on the least significant bits. An entry cannot span across multiple longs; instead, padding is inserted as required, starting from the most significant bits.

For example, assuming a bits per block value of 15, and that bit 0 is the least significant bit, the data is stored such that bits 0 through 14 are the first entry, 15 through 29 are the second, and so on. The fourth entry ends on bit 59, and since only 4 bits are left, they become padding, and the fifth entry starts on the next long.

Note that since longs are sent in big endian order, the least significant bit of the first entry in a long will be on the *last* byte of the long on the wire.

 ⚠ This format was changed in Minecraft 1.16. In prior versions, entries could cross long boundaries, and there was no padding.

Visual example [edit | edit source]

5 bits per block, containing the following references to entries in a palette (not shown): 1 2 3 4 5 6 6 4 8 0 7 4 3 1 3 1 5 1 6 9 1 4 1 0 1 2 0 2

 00 20 86 31 48 4 1 8 8 41
 0000 0000 0000 0100 000100 001100 001100 001001 001000 001000 000011 000100 001000 000001
 0000 0000 0000 0100 000100 010011 001100 010011 000000 011111 011011 000111 001000 001111

Hints for implementers [edit | edit source]

The number of entries per long may be calculated as floor(*64* / *bits_per_entry*). The number of longs in the array may then be calculated as ceil(*number_of_entries* / *entries_per_long*). It may seem like this calculation could be simplified as number_of_entries times *bits_per_entry* divided by 64, but that formula is incorrect, since the intermediate rounding is significant and accounts for the padding at the end of each long.

In languages that lack a ceiling division operator, one can do the following:

```
// assuming integer division rounds towards 0, which is usually the case.
entries_per_long = 64 / bits_per_entry
number_of_long = number_of_entries + (entries_per_long - 1) / entries_per_long
```

The index of the long an entry is located in may be calculated as floor(*entry_index* / *entries_per_long*), and the 0-based index of the least significant bit of the entry within the long as (*entry_index* % *entries_per_long* * *bits_per_entry*) (where % is the remainder operator).

To read an entry in a language with C-like bitwise operators, one can do the following:

```
// in some languages it is necessary to cast the 1 to a 64-bit type first, or else
// the shift will be performed in 32 bits, a C-style cast is shown as an example.
entry_mask = ((uint64_t)1 << bits_per_entry) - 1
long_index = entry_index / entries_per_long
bit_index = entry_index % entries_per_long * bits_per_entry
value = (data_array[long_index] >> bit_index) & entry_mask
```

And to write an entry:

```
data_array[long_index] &= ~(entry_mask << bit_index)
// if value has a smaller integer type, it may again be necessary to cast it to 64 bits.
data_array[long_index] |= ((uint64_t)value << bit_index
```

Note that there are more efficient ways to convert all entries in a section to/from this format, as well as more efficient ways of doing the division and remainder computations for individual accesses with specific *bit_per_entry* values. This code merely represents the simplest possible implementation.

Tips and notes [edit | edit source]

There are several things that can make it easier to implement this format.

- Servers do not need to implement the local palette initially (instead always using 15 bits per block), although it is an important optimization later on.
- The Notchian server implementation does not send values that are out of bounds for the palette. If such a value is received, the format is being parsed incorrectly. In particular, if you're reading a number with all bits set (15, 31, etc), you might be reading skylight data (or you may have a sign error and you're reading negative numbers).
- The Notchian client generally does not render chunks that lack neighbors. (As of 1.20.2 such chunks appear to sporadically become visible anyway, and do so consistently when interacted with.) This means that if you only send a fixed set of chunks with no empty chunks around them, then some of them will not be visible, although you can still interact with them. This is intended behavior, so that lighting and connected blocks can be handled correctly.

Full implementations [edit | edit source]

- Java 1.12.2, writing only with palette^{[?]</}